ScratchBPU: Exploiting Speculative Updates in Intel Branch Prediction Units

Abstract

Modern CPUs rely extensively on Branch Prediction Units (BPUs) to sustain high instruction throughput. This work reveals that the BPU in modern Intel CPUs is updated with the branch's target address after the execution of a branch, but before the branch is architecturally committed. This means that speculative branch targets are written into shared BPU structures. This establishes a new attack vector, exploited by *ScratchBPU* – a class of Branch Target Injection (BTI) attack primitives that inject and exfiltrate arbitrary data directly through BPU structures to create a high-bandwidth covert channel. These primitives also enable a side-channel attack that leaks secret-dependent cryptographic data from a victim, breaking process isolation. Furthermore, we release an automated gadget-discovery tool to identify *ScratchBPU*-vulnerable code patterns in binaries. Finally, we propose a hardware-only, area-neutral mitigation that eliminates speculative BPU updates, achieving performance parity with an insecure baseline while fundamentally blocking the established poisoning vectors.

1 Introduction

Modern processors rely heavily on predictors to avoid costly pipeline stalls in the presence of control flow [28, 37, 55]. These include predictors targeted at conditional branches [38, 46, 49], indirect (computed) branches [31, 48], calls and returns [35], etc. Because these structures are often shared across protection domains, process boundaries, and privilege levels [33, 43, 78], they have become a recurring source of microarchitectural attacks [19, 29].

One class of attacks uses the branch predictor as a *mistraining mechanism*. Spectre-V1 [29] mistrains the conditional branches to bypass intended control-flow checks, Spectre-V2 [29] poisons indirect branch predictors to perform branch target injection (BTI), and return-based variants such as ret2spec [35] and Spectre-RSB [30] exploit the Return Stack Buffer (RSB). Subsequent attacks further expand this surface by exploiting branch-history state, return prediction, and transient training, including BHI/Spectre-BHB, Retbleed, Inception, and branch privilege injection [11,43,60,71,72,77]. Those attacks inject conditional branch outcomes or branch targets into the Branch Prediction Unit (BPU), but only for the purposes of poisoning branches to redirect execution to code that can leak data [29, 33, 36].

A second class of attacks uses branch predictor state itself as the leakage medium. Prior work has shown that BTB state can leak address-layout and control-flow information [18, 32], while conditional-predictor attacks can infer branch directions or recover control-flow history [19, 27, 77]. However, these attacks are fundamentally tied to conditional control flow: they either reveal direction bits, infer path history, or leak information already encoded in branches executed by the victim [19, 27, 77]. Recent work showed that, on the Intel processors studied, even wrong-path conditional branches can leave persistent traces in the conditional branch predictor (CBP) [27].

Our work shows that wrong-path predictor updates are not limited to the CBP: they also affect target-prediction structures such as the indirect branch predictor (IBP) and branch target buffer (BTB), with potentially serious ramifications, in part because the fidelity of those structures (48 bits or 32 bits of branch address per entry, respectively) is so much higher. This, combined with the ability to invisibly execute arbitrary code sequences in speculation to fill those entries, presents a potent combination [60, 72].

This paper introduces *ScratchBPU*, which uses the BPU as *scratchpad memory*, directly translating secret values across process boundaries. This separates it from prior attacks which poison the BPU only to redirect control flow (e.g., to a data leakage gadget) [29,33,36]. We develop primitives that enable the write of arbitrary secret values into the BPU, either in a covert process or an unsuspecting victim, that can check for the presence or absence of particular values in a BPU entry, and that can read a large arbitrary value from the BPU. We demonstrate both a covert channel and an end-to-end side channel communicating a secret value from cryptographic

code.

We start by carefully reverse engineering the Intel Raptor Cove IBP and BTB branch predictors, analyzing the existence and the precise properties of this vulnerability, demonstrating that values written into the BPU persist across process switches and can be made to persist arbitrarily long in the BPU, even after many executions of speculative branches intended to exfiltrate the value.

Another key contribution of this paper is an extended demonstration and analysis of an ironclad hardware defense that can be implemented with minor architectural modification. By delaying BPU update to commit, we inject no speculative values into the BPU. Conventional wisdom holds that such a delay prevents the predictor from positively learning from wrong-path execution, and delays updates that might be useful for the next branch instance in a tight loop. However, our results debunk this, demonstrating a speculative-update-free predictor that actually outperforms current insecure branch predictors.

2 Background

2.1 Intel Branch Prediction Unit

This section covers the relevant components of the Branch Prediction Unit (BPU) in modern Intel CPUs [25, 39].

Path History Register: The Path History Register (PHR) is a shift register that tracks the global history of branches. Half&Half [78] describes how the PHR is formed and how it indexes the entries of the Conditional Branch Predictor (CBP) of recent high-end Intel microarchitectures. Further work shows how it is also used to index the Indirect Branch Predictor (IBP) [33]. In this work, we use primitives to manipulate the PHR to achieve precise control of the IBP, similar to those presented in [77] and [78].

Conditional Branch Predictor: The Conditional Branch Predictor (CBP) is responsible for predicting whether a conditional branch is taken or not based on the history of the branch and other branches [50, 78]. Intel's CBP is based on the TAGE branch predictor [50, 51, 78]. BranchSpec [27] documents that the PHR is updated during speculation and the information of branches taken on the wrong path remains encoded within the conditional branch predictor after speculation ends.

Indirect Branch Predictor: In normal program execution, it is common for control flow to be directed to targets which are computed at run time. To prevent front end stalls in these situations, CPUs rely on an Indirect Branch Predictor to predict the jump target address [33, 48]. The predictions are based on history and are continually updated on each new indirect jump [33, 48]. Upon detecting a mispredict, the speculatively executed instructions are squashed in the Re-Order Buffer (ROB) and proper execution resumes from the correct target [29]. Intel CPUs use a structure known as ITTAGE [47, 48] in which the prediction entries are divided

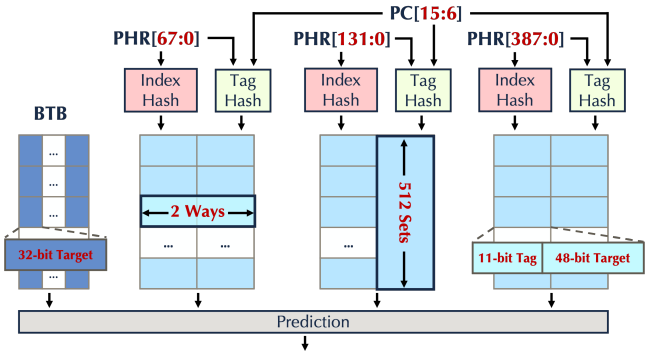


Figure 1: Structure of the Indirect Branch Predictor (IBP) with the Branch Target Buffer (BTB) serving as the base predictor.

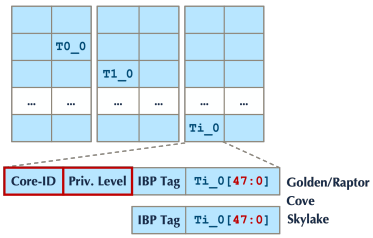


Figure 2: Structure of a single entry inside the IBP with different metadata values highlighted.

into a set of associative structures similar to caches [33]. The index and tags of the IBP are formed from a hash of the PHR and the branch's PC [33].

For Raptor Cove cores, bits [15:6] of the jump instruction's PC are used to index into the IBP entry with the PHR serving as the tag. Each entry holds the full 48 bits of the indirect jump's target. Figure 1 illustrates the full IBP structure. Each entry also comes with certain metadata shown in Figure 2 which is used to accept or reject predictions provided by the IBP. Intel uses these metadata values to harden the IBP against transient execution attacks in more recent processors [33]. In Section 4 we examine the effectiveness of these defenses against *ScratchBPU* primitives.

Branch Target Buffer: When a jump instruction is fetched, the CPU must wait for the entire instruction to be decoded to determine the type of instruction and, in cases of direct jumps, the target address. The Branch Target Buffer (BTB) is a structure that remembers previously encountered branch instructions and their targets [31, 33]. For indirect branches, the BTB acts as another source of target predictions [33]. In the case of indirect branches, the BTB serves as the base level predictor for the IBP and stores the lower 32 bits of the jump target on Raptor Cove CPUs [33]. The upper 16 bits are assumed to be the same as that of the jump instruction. Crucially, only bits [23:0] of the jump instruction's PC are used to index into the BTB and the PHR is not used in accessing the BTB entries.

Return Stack Buffer: The Return Stack Buffer (RSB) is a fixed size microarchitectural buffer that stores predictions for `ret` instructions [36, 74]. For each `call` instruction, the address of the next instruction after the call is pushed onto the RSB. Upon encountering a `ret`, this address is provided as a prediction and the entry is popped from the stack [36, 74].

2.2 Transient Execution Attacks

Transient execution attacks are a class of microarchitectural side channel attacks in which an attacker can leak information from the footprint left behind by speculatively executed instructions [16, 29, 29, 34, 42, 54]. In Spectre-V1, an attacker relies on training the conditional branch predictor and later causing a mispredict to create a large window of speculation. Within this window, a secret dependent load instruction is executed that leaves behind a cache footprint. This footprint is observed via a flush and reload [76] or a prime and probe [41] of the cache to leak the secret dependent value.

In Spectre-V2, also known as Branch Target Injection (BTI), an attacker directs the flow of speculative execution towards consequential instructions to later observe their effects via side channels. This is achieved by poisoning the entries of the BPU via aliased branches with attacker controlled values, which will later serve as predictions for an indirect jump or call instruction in a victim process. Related work has demonstrated this attack surface against SGX enclaves and through decoder-detectable mispredictions [17, 73, 80].

Spectre-RSB [36] exploits the RSB's limited entries by making deeply nested calls which upon unwinding result in an RSB underflow. That triggers predictions from the BTB which can be poisoned via BTI.

Like other transient attacks, ScratchBPU exploits unexpected behavior during speculation, and state left in microarchitectural structures after speculation ends [45, 65]. However, it uses the IBP (or the BTB) as a conduit for leaking data directly, rather than a conduit for poisoning control flow.

3 Understanding the BPU Update Policies

The ScratchBPU attack is enabled by a vulnerability in the update policies of the BPU. This section, then, seeks to uncover if, how, when, and for what range of values the BPU structures are updated under speculation, with particular focus on the IBP.

All our experiments were performed on a 13th generation Intel i9-13900KS performance core (Raptor Cove). The operating system used is Ubuntu 22.04.5 LTS, running an unmodified Linux kernel version 6.12.10. This machine also serves as our Host for our Host Machine and Guest Virtual Machine (VM) experiments, where we use KVM as our hypervisor and Qemu to run the VMs. The VMs run Ubuntu 24.04.3 LTS operating system. All virtual machines are configured to run on the same logical core of the SMT-enabled performance core, so that all their processes are architecturally co-located on the same core id.

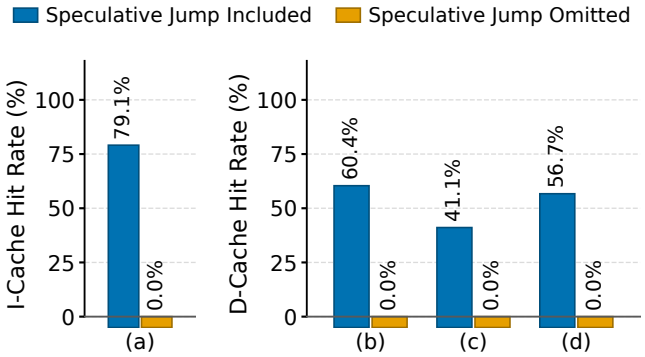


Figure 3: Variation of Cache hit Rate when the speculative jump I_{write} is included and omitted when attempting to write to the IBP in different settings. (a) Measures I Cache hit rate for the target within the same process. (b) Measures D Cache hit rate for a byte loaded under the target within the same process. (c) Measures D Cache hit rate for a byte loaded under the target when writing and reading is across processes and the target is legal in both. (d) Measures D Cache hit rate for a byte loaded under the target when the writing and reading is across processes and the target is legal only when reading.

We use the following Intel performance counters [1, 26] to measure hardware events in our reverse engineering experiments – `BR_INST_RETIRED.INDIRECT`, `BR_MISP_RETIRED.INDIRECT`, and `FRONTEND_RETIRED.L2_MISS`. Our test harness is derived from the reverse engineering harness provided by Agner Fog [20, 63, 64]. Section 6, however, shows that the time stamp counter is sufficient for *ScratchBPU* primitives to construct attacks.

To reverse engineer the BPU update policies, we conduct microarchitectural tests on specific IBP entries. We ensure the prediction originates from the IBP by making the BTB see multiple targets for each indirect branch; this ensures that upon an IBP tag hit, the IBP provides the prediction [33]. We study BTB update policies separately in Section 3.1. Most of these tests involve a kernel with the following four elements:

- **Step 1:** Set the PHR to a known value $\mathbf{PHR_{known}}$, and speculatively execute an indirect jump $\mathbf{I_{write}}$ to a target $\mathbf{T_{secret}}$
- **Step 2:** Flush $\mathbf{T_{secret}}$ from the cache
- **Step 3:** Set the PHR to $\mathbf{PHR_{known}}$ and architecturally execute an aliased detector indirect jump $\mathbf{I_{write}}$ to target $\mathbf{T_{arch}}$
- **Step 4:** Check if the BPU provided $\mathbf{T_{secret}}$ as a prediction that results in the frontend caching it.

Listing 1 illustrates these four steps in our experimental code.

```
; Step 1
... ; Train the je at C to be not taken
mov PHR, PHR_known
mov [target_ptr], T_secret
mov [condition_ptr], 0 ; Make the je branch
    architecturally taken, so that a misprediction
     occurs
clflush [condition_ptr] ; Create a large enough
    speculative window for I_write
call branch_gadget

; Step 2
clflush [T_secret] ; Flush T_secret
clflush [secret_byte] ; Flush secret_byte

; Step 3
mov PHR, PHR_known
mov [target_ptr], T_arch
mov [condition_ptr], 1 ; Make the je branch
    architecturally not taken, so that I_write will
    be architecturally executed
call branch_gadget

; Step 4
CHECK_CACHED(T_secret) ; Reload T_secret
CHECK_CACHED(secret_byte) ; Reload secret_byte
; If hit, I_write during Step 1 successfully updates
    IBP under speculation
...
branch_gadget:
    cmp [condition_ptr], 0
C:
    je T_return ; Trained to be not taken
I_write:
    jmp [target_ptr]
T_arch:
    nop
T_return:
    ret
...
T_secret:
    mov rcx, secret_byte
    ret
```

Listing 1: Experimental setup to reverse engineer the IBP update policy

We use a write-to-PHR primitive, similar to previous work [77, 78], to set the PHR to a known value. To create the speculation needed for the first step, we train a conditional branch **C** as not taken before flipping the condition. The indirect branch $\mathbf{I_{write}}$ is placed in the not taken path, and architecturally jumps to $\mathbf{T_{arch}}$ during training. Finally, we call the branch target with the conditional branch taken, and the jump target (which should never be executed) changed to $\mathbf{T_{secret}}$.

These four steps are sufficient for our first experiment. If the BPU was updated during speculative execution, the next time we architecturally execute this jump to $\mathbf{T_{arch}}$, we will speculatively jump to $\mathbf{T_{secret}}$ before it is resolved with the correct target. From Figure 3 (a) we see the evidence that the speculative target has been written. In the absence of the speculative write of $\mathbf{T_{secret}}$, the instructions at $\mathbf{T_{secret}}$ are not cached; but when we include the speculative indirect jump, then we find $\mathbf{T_{secret}}$ cached.

Observation 1. The IBP is updated under speculation and the updated value remains in the IBP after speculation.

Next, we check whether the target instructions at the written target are also speculatively executed (the I Cache result only shows that they were fetched). As illustrated in Listing 1, we add a move instruction below our written target $\mathbf{T_{secret}}$ to load a byte from memory that was previously flushed from all caches.

Figure 3 (b) shows that the miss rate for the previously flushed byte drops when the target $\mathbf{T_{secret}}$ is speculatively fetched. This tells us that speculatively updated values are treated identically to architecturally updated values by the hardware. The hardware begins to fetch and execute instructions at the written target and the number of instructions executed depends on the size of the speculation window created.

Observation 2. The speculatively updated value is treated similarly to an architecturally updated value.

In our attack, we want to use the contents of the IBP as scratchpad memory for the communication of values. Any read of that value will involve execution of a jump that indexes to that IBP entry. If we want to read the full value (or even a large portion of the bits), we either need to be able to capture all of the bits with a single execution, or be able to read it multiple times. The complication is that our read, as described, is destructive – an architecturally executed branch will speculatively branch to the secret, but then when resolved will overwrite the IBP entry with the actual target.

Doing multiple reads, then, would require continually reseeding the value into the IBP. For some targets, that is possible – e.g., continuously attacking a static cryptographic key, attacking an enclave that can be repeatedly restarted, etc. However, an attack that only requires the victim to be probed once has far greater application and is more dangerous.

In our first experiment, we execute Steps 3 and 4 repeatedly, and find that the vast majority of time we only read the secret successfully a single time. This confirms that architecturally executing the branch results in a destructive read, after which the IBP entry is overwritten. The blue line in Figure 5 illustrates this.

Next, we attempt to devise a non-destructive read of the IBP. In Step 3, we put the original detector indirect branch inside the speculation window of an enclosing mispredicted branch. The enclosing branch may be either a conditional branch or an indirect branch. Therefore, we can execute our detector branch speculatively instead of architecturally. But that is not sufficient, for the same reason the vulnerability exists in the first place – as soon as the speculative detector indirect branch resolves, the IBP is overwritten (Figure 4), not waiting for commit. Thus we need two speculative windows

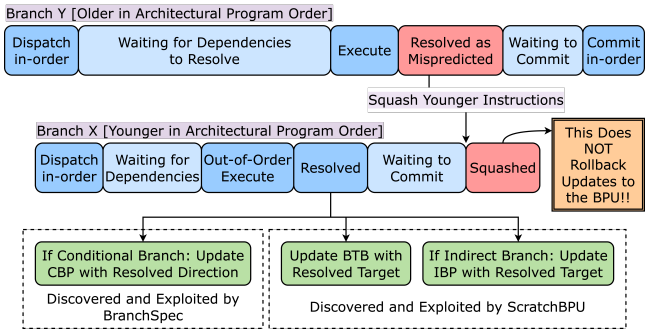


Figure 4: This figure shows how the branch predictor is updated when the branch is resolved (prior to commit), and how that results in updates from the wrong path to the BPU, as confirmed by our results.

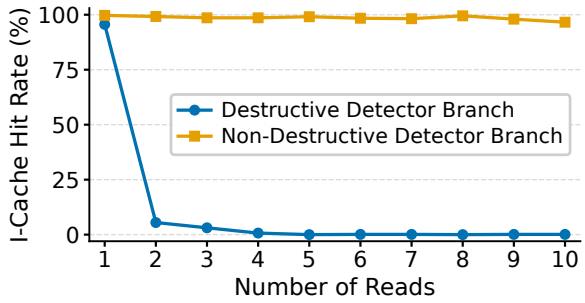


Figure 5: I Cache hit rate of $\mathbf{T_{secret}}$ in Step 4 as the number of detector reads increases.

of different latencies to make this work. The enclosing mispredicted branch (only required to enter wrong-path execution) should be resolved with a moderate delay (long enough to reach the indirect branch and execute it). But the detector indirect branch must resolve with a longer delay, such that the enclosing branch always resolves first. In that way, the indirect branch never resolves its target and never writes into the IBP.

For this new experiment, we use a conditional branch as an enclosing branch and flush the condition from the caches, but make the indirect branch depend on a chain of four loads (all flushed from the caches). With this experiment we are able to recover the written value for multiple times without overwriting it. As shown in Figure 5 (orange line), during Steps 3 and 4, when we use the non-destructive detector branch to test whether $\mathbf{T_{secret}}$ has been written into the IBP, $\mathbf{T_{secret}}$ is still speculatively fetched after multiple detections. This indicates that the original IBP entry remains intact and is not overwritten by the detector branch. In our experiments, the same written value can still be recovered after more than 1000 repeated reads. This enables us to scan the IBP non-destructively, without needing to reinject $\mathbf{T_{secret}}$ before each read.

```
; Write Process
... ; Train the enclosing je at C to be not taken
mov PHR, PHR_known
mov [target_ptr], T_secret
mov [condition_ptr], 0
clflush [condition_ptr]
call branch_gadget
...
branch_gadget:
    cmp [condition_ptr], 0
C:
    je T_enclosing  ; Architecturally taken while
    speculatively not taken
I_write:
    jmp [target_ptr] ; Speculatively update IBP
    with T_secret
T_enclosing:
    ret
...
T_secret:
    ...
    ret
```

Listing 2: Experimental setup highlighting the Write Process

Observation 3. A resolved detector branch destructively reads the IBP by overwriting the injected value. If the detector branch is squashed by an enclosing speculative branch before it resolves, it can repeatedly read the IBP without updating it.

Having confirmed that we can read and write under speculation to the BPU, we further explore persistence of those reads/writes – i.e., the conditions under which the value can still be read. First, we explore the cross-process boundary. For this experiment, we set Indirect Branch Prediction Barrier (IBPB) to be in the default conditional mode. We believe that the BPU entry metadata does not store any process ID bits [33]. If this experiment succeeds, it will confirm that, but also confirm that in the default IBPB setting the IBP is not flushed on process context switch.

We have two processes, called the **write process** and the **read process**. We write a value into the IBP under speculation from inside the write process and see if we can non-destructively read the value written from inside the read process. The write process and read process are illustrated in Listing 2 and Listing 3 respectively. We see the same basic steps as in the previous sections, the only major difference being that the speculative write occurs in a process different from where the read occurs.

Crucially, we ensure that the target $\mathbf{T_{secret}}$ is a valid address in the address spaces of both the write process and the read process. We also ensure that both processes are co-located on the same logical core id. Recall from Figure 2 that the IBP entry stores the core ID to prevent poisoning based attacks from a different SMT context. From Figure 3 (c), we see that the value written in one process can be read from another process, as the address corresponding to the secret is cached in the read process nearly half the time. The signal is a bit

```
; Non-Destructive Read Process
... ; Train the enclosing je at C to be not taken
mov PHR, PHR_known
mov [target_ptr], T_detector
mov [condition_ptr], 0
clflush [target_ptr] ; Delay the resolution of
    I_read
clflush [T_secret_aliased] ; Flush T_secret_aliased
call branch_gadget
CHECK_CACHED(T_secret_aliased) ; Reload T_secret_aliased
...
branch_gadget:
    cmp [condition_ptr], 0
C:
    je T_enclosing ; Architecturally taken while
    speculatively not taken
I_read:
    jmp [target_ptr] ; Target resolution delayed,
    non-destructive IBP read without updating IBP
T_detector:
    nop
T_enclosing:
    ret
...
T_secret_aliased: ; Aliased with T_secret in virtual
    address
    ...
    ret
```

Listing 3: Experimental setup highlighting the Read Process

weaker, but still strong.

Observation 4. We can read the value written into the BPU during speculation across process boundaries if both the processes are co-located on the same logical core.

Next, we seek to understand if there are any restrictions on what values can be written into the 48-bit IBP. For example, if the process of resolving a branch in the pipeline involves any kind of permission check (e.g. for execution privilege), it would restrict what values can be written. For this, we repeat the previous experiment while ensuring that $\mathbf{T_{secret}}$ is an illegal address within the write process, but legal within the read process (so that we can detect it).

Figure 3 (d) shows us that we still manage to successfully fetch the address $\mathbf{T_{secret}}$ inside the read process. Thus, there is no known limit to what can be written into the IBP – it doesn't matter if the writer has execute access, read access, or whether the address is mapped at all; presumably any 48-bit value can be passed in.

Observation 5. It is not necessary for the value being written under speculation to be a legal branch address. We can write arbitrary values into the BPU.

Lastly, we want to definitively understand when the IBP is written. Returning to the code in Listing 1, we experiment with the relative latencies of condition **C** (which defines the length of the total speculation window), $\mathbf{I_{write}}$ (which determines when the branch instruction can be fetched and possibly executed), and $\mathbf{T_{secret}}$ (which determines when the target can be resolved for the indirect branch).

Table 1: BPU Update Status Based on Resolution Timing

Resolution of $\mathbf{I_{write}}$	Resolution of $\mathbf{T_{secret}}$	BPU Update
Before Speculation Ends	Before Speculation Ends	✓
Before Speculation Ends	After Speculation Ends	✗
After Speculation Ends	Before Speculation Ends	✗
After Speculation Ends	After Speculation Ends	✗

Within the experiments, we force the start and end of these stages to occur at specific times by selectively flushing the elements on whose resolution the stages depend. Table 1 illustrates our ability to write to the BPU when these elements are flushed or retained in the cache. From these results, it becomes clear that as long as the branch instruction $\mathbf{I_{write}}$ and the value to be written $\mathbf{T_{secret}}$ are resolved before the end of speculation, we can write to the BPU. However, the indirect branch is removed immediately upon the end of the speculation window (the resolution of the conditional branch) and has no ability to write to the BPU after that.

3.1 Writing into the BTB Under Speculation

So far, we have focused on understanding the behavior of the IBP when it is subjected to speculative writes. We now turn our attention to the BTB to understand its behavior.

Writing into the BTB is more straightforward than writing to the IBP, as addressing a BTB entry does not have the complex indexing based on the PHR. In these experiments, to ensure that we are measuring BTB effects rather than IBP effects, we force the addresses of the writing jump and the reading jump to alias in the BTB but have different PHRs. Figure 6 illustrates the results of trying to write into the BTB. We still see a signal in all cases, although weaker than the IBP and much weaker (but still definitely a signal) in the cross-process cases. We believe this is a function of the noise caused by intervening branches during a context-switch having far more impact on the BTB – again, due to its much simpler indexing function.

Observation 6. Observations 1, 2, 4, and 5 also apply to the BTB.

4 Efficacy of Intel Defenses

In this section, we examine the efficacy of different Intel defenses against our *ScratchBPU* primitives. Understanding

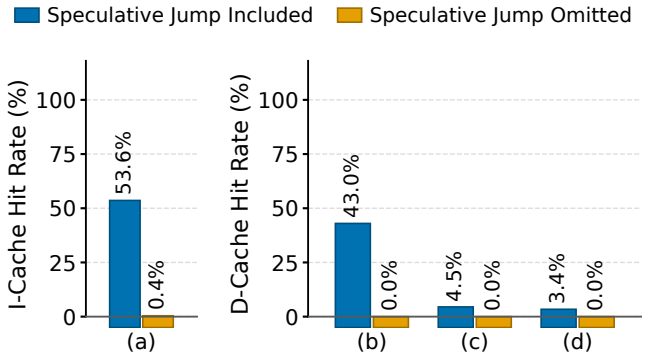


Figure 6: Cache hit rates when we write into and read from the BTB under similar scenarios to Figure 3.

the constraints placed on our attack primitives will help us define our threat model and construct our attacks.

4.1 Control Flow Enforcement Technology

Intel Control Flow Enforcement Technology (CET) is a defense against control flow hijacking attacks which uses indirect branch tracking capabilities to provide software the ability to restrict Call Oriented Programming (COP) and Jump Oriented Programming (JOP) attacks [53] [21] [15] [52].

We see if enabling Intel CET prevents our speculatively written value from being fetched. We repeat the experiment from Section 3 now compiled with Intel CET enabled. We observe that speculatively written target is fetched and only the first instruction after the target label is executed. This is in line with Intel's claims that CET allows at most one speculative instruction to execute [25]. This is sufficient for any of our read primitives described later to succeed.

Observation 7. Speculatively written targets (to invalid locations) are still fetched and one instruction below the target is executed when Intel CET is enabled.

4.2 Single Thread Indirect Branch Predictors

Single Thread Indirect Branch Predictors (STIBP) is an indirect branch control mechanism that restricts the sharing of branch prediction between logical processors on a core [2,24].

From Observation 4 we know that written values can be read across co-located process boundaries. We test to see whether STIBP protections are compromised with *ScratchBPU*. We repeat the same experiment as in Section 3 except now the write process and read process run on different cores. Figure 11 in the Appendix illustrates that the written target is not fetched when the write and read processes run on different cores. This necessitates that both the processes be co-located on the same core. We observe that we cannot read the value across process boundaries even when the processes run on sibling SMT cores that are on the same physical core. This observation is consistent with the results of previous works that demonstrate that IBP entries store the logical core id value as part of an entry's metadata [33].

Observation 8. Both the write and read processes must be co-located on the same logical core. Otherwise, we cannot read the written value across process boundaries.

4.3 Enhanced Indirect Branch Restricted Speculation

Indirect branch restricted speculation (IBRS) is an indirect branch control mechanism that restricts speculation of indirect branches across privilege domains [6,24]. Intel specifies that their CPUs recognize four distinct predictor modes, from most privileged to least privileged, these are: 1. Host Supervisor, 2. Host User, 3. Guest Supervisor, 4. Guest User

eIBRS is meant to protect the entries updated in one mode from affecting predictions in another.

To test if speculatively updated values also fall under this guarantee, we run the write process and read process within different domains to see if we are able to read the written value. Through these experiments, we confirmed that written values cannot be read across privilege domains. This observation matches observations made in [33] that the entry metadata has bits to store the privilege domain of a branch. These observations are also in line with observations made in [43] which confirms that the metadata values are provided by the domain writing into the BPU.

Observation 9. Values written into the BPU in one privilege domain cannot be read in another privilege domain.

4.4 Indirect Branch Predictor Barrier

Indirect Branch Predictor Barrier (IBPB) is an indirect branch control mechanism that flushes the entire IBP on context switches within the same privilege level [5,24,56], eIBRS and IBPB together are meant to harden the Intel BPU from all forms of BTI. However, flushing on every context switch adds massive performance penalties. Intel's compromise is to perform conditional flushes. In our following experiments, we explore the meaning of conditional flushes.

Recall from Observation 4 that process context switches do not trigger an IBP flush. Next, we run the same write and read processes across two VMs. Both VMs are configured to run on the same performance core. Figure 12 in the Appendix

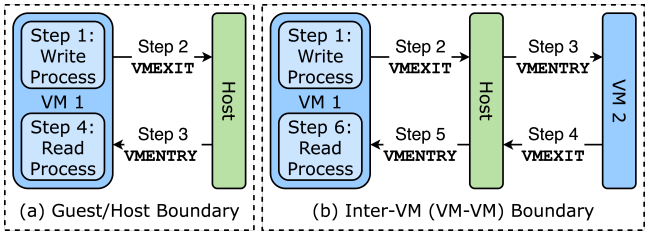


Figure 7: Experimental setup to validate IBP state persistence across virtualized context transitions. In both configurations, write and read operations are performed within the same guest VM, separated by an intervening exit to (a) the host or (b) a secondary guest VM.

shows that the written target is fetched by the read VM only when IBPB is switched off. When IBPB is set to the default conditional setting, we are not able to read the value written in one VM from another VM. This tells us that conditional IBPB flushes the IBP on either `VMEXIT` or `VMENTER`.

To understand whether it is `VMENTER` or `VMEXIT` that causes the flush, we run both the write and read processes within the same VM, but exit from the VM to the host in the write process and re-enter the VM from the host into the read process of the VM. Figure 7 (a) illustrates this setup. In this experiment, we see that the written target is always fetched by the read process. This tells us that `VMEXIT` does not cause the IBP flush.

Next, we repeat the same experiment, but enter a second VM between writing a value and reading a value. The sequence is depicted in Figure 7 (b). However, we now see that the written value is never fetched by the read process. This confirms that it is `VMENTER` that causes the IBP flush. Moreover, the flush operation is triggered only if we exit one VM and enter another one.

Observation 10. VMENTER causes the IBP to be flushed. The flush is not triggered if we exit and re-enter the same VM.

5 Threat Model

This section introduces the threat model in this paper.

We consider an unprivileged attacker who can co-locate their process on the same logical core as their victim workload. The attacker does not require elevated privileges, kernel access, or physical access to the system. Specifically, the attacker:

- Runs user-level code on the same logical core [58, 61].
- Can allocate and access memory through standard OS mechanisms (e.g., malloc and mmap).
- Can track and manipulate the value of the PHR.
- Has sufficient knowledge of the victim process's address space to locate vulnerable indirect branch instructions.

Notably, in our chosen attack setting, our attacker is a separate process that is simply co-located on the same logical core as a passive victim process.

To track and manipulate the value of the PHR we rely on primitives established in [77], which can not only set the PHR, but recreate the PHR of a victim with high accuracy at arbitrary points of execution. Further, we do not explicitly break ASLR in our demonstration as this has already been accomplished by other means [22].

Our ideal victim would be a cryptographic routine which stores some secret value inside a register at the time of an indirect jump.

Our end to end attack demonstrates attacking the OpenSSL cryptographic function `PKCS5_PBKDF2_HMAC`. This function derives a key from a plaintext password by using the password as the HMAC key and iterating HMAC over a salted counter thousands of times.

6 Attack Primitives

The goal of *ScratchBPU* is to use the BPU as shared value storage (memory) for processes that are not allowed to share memory. To that end, we will compose our attacks from three primitives:

- ScratchBPU Write: Writes an arbitrary value to the BPU
- ScratchBPU Check: Checks a BPU entry for the presence or absence of a known value.
- ScratchBPU Read: Exfiltrates an arbitrary value written to a BPU entry.

6.1 ScratchBPU Write Primitive

The write primitive typically needs to execute in the victim space, and thus requires us to piece together a gadget from victim code. We have already seen in Section 3 how a value written in one process can be read in another process. However, we are unlikely to find gadgets of an indirect jump on a register containing a secret dependent value under a conditional branch. However, recent work has shown the ability to gain precise control over the control flow of a victim process [33, 77]; thus, we merely need to find any speculative control flow path (whether intended or even architecturally possible) from the writing of a secret to a register to a use of that register in an indirect branch.

Listing 4 shows one possible example – this could be a call to a shared library function or a call involving runtime polymorphism at a time when any register holds a secret value.

In this new setting, the three components of our update timeline are as follows

- **Speculation**: Speculation is created by causing $\mathbf{I}_{\mathbf{speculation}}$ to mispredict with the help of BTI

```
mov PHR, PHR_known
call secret_handling_function

secret_handling_function:
    mov rax, secret
    mov rdx, T_arch
I_speculation: ; This will incorrectly jump to
    branch_gadget under speculation
    jmp rdx
branch_gadget:
I_write:
    jmp rax
T_arch:
    nop
    ret
```

Listing 4: The BPU write primitive showing how to create speculation under an indirect branch and use it to write to the BPU.

- **Indirect Jump Instruction**: Under speculation, we use currently available techniques [29, 33] to steer execution towards another indirect jump $\mathbf{I_{write}}$ that jumps on $\mathbf{reg_{secret}}$
- **Indirect Jump Target**: Here, the target of our jump $\mathbf{I_{write}}$ is the secret value $\mathbf{T_{secret}}$ that gets written to the BPU. Again, this need not be a legal value.

Following this sequence, the lower 48 bits of the secret (32 bits if using the BTB) are in the IBP.

6.2 ScratchBPU Check Primitive

Before exfiltrating an unknown value from the BPU, we provide a primitive to check if a known value was written to the BPU. Assume that $\mathbf{T_{known}}$ was conditionally written to the BPU using the *ScratchBPU* write primitive with the setup in Listing 4. To check if $\mathbf{T_{known}}$ was written, the attacker's read process must satisfy two requirements. First, the read process must contain an indirect branch $\mathbf{I_{read}}$ that indexes into the same BPU entry as $\mathbf{I_{write}}$. Second, $\mathbf{T_{known}}$ must be a legal instruction address within the code section of the read process.

Once these two requirements are met, the attacker flushes the address $\mathbf{T_{known}}$ from the instruction cache and executes the jump $\mathbf{I_{read}}$ ensuring its architectural target also takes time to resolve. During the resulting speculation, the hardware will incorrectly predict $\mathbf{T_{known}}$ as the target and fetch it back into the cache. Finally, by measuring if $\mathbf{T_{known}}$ is cached, the attacker can check if this was the value that was written previously.

Note that the *Check* primitive is easily extended – e.g., a secret that can take one of 8 or 16 values is easily searched with a series of check primitives. In fact, using Observation 3, this can be done with a single write, and N checks in the attacker space.

6.3 ScratchBPU Read Primitive

Reading an unknown value written into the BPU is more challenging as the target to which $\mathbf{I_{read}}$ speculatively jumps is anywhere within the entire `x86_64` address space of 256 terabytes. Even if we restrict ourselves to user space addresses (i.e., the lower 47 bits of the secret), we have a search space of 128 terabytes that cannot be searched tractably.

However, any byte that is not cached must be fetched from memory and this involves translating the virtual address of the memory to its physical address. The hardware caches common translations within special caches called Translation Lookaside Buffers (TLBs). Modern processors also cache partial translations within special caches called page walker caches [66] [25]. Peek-a-Walk [69], shows how dereferencing a data pointer under speculation leads to a partial page walk. The progress of a page walk from a higher level to a lower level is determined by whether or not the lower level page table is located at a valid address. Therefore, by selectively mapping address regions to a process's address space, an attacker can control the progress of a page walk and in doing so progressively leak the virtual address bits of the pointer being de-referenced. Each time a new page table entry is discovered in the cache, we leak 9 bits. 3 bits corresponding to the cache offset of the previous level, and 6 bits corresponding to the cache line of the current level.

Peek-a-Walk reveals a speculative data pointer that is being de-referenced. For our purposes, we modify this attack to leak the address of a speculative instruction that $\mathbf{I_{read}}$ tries to fetch and execute under speculation. As part of this modification, we flush the instruction caches and TLBs before calling $\mathbf{I_{read}}$ to force a page walk. Also, execute permissions are provided to the memory regions we map and the entire mapped region is filled with single byte `ret` instructions. Each new page table entry is unmasked by priming and probing the data cache, and the final page fetches the byte into the instruction cache.

Using this method, we leak higher order bits of the written value sequentially for each page table entry discovered, until we arrive at the last level. At this point, we have recovered 33 bits from levels 4, 3, and 2 of the page walk. The probe of level 1 should give us 3 more bits (the rest of the bits map to the same page and are not accessible with this method); however, we found the probe of level 1 to be too noisy, so we do not use it – leaving us with 15 unknown bits in the least significant positions.

Leaking bits 14-6: At this point we know the top 33 bits of our 48 bit address. We know that the flush and reload primitive signals whether a previously flushed cache line was reloaded under speculation. Hence, we map targets that all have the same top 33 bits as our leaked value and are separated by a cache line boundary. This leaves us with 512 targets. By performing flush and reload on these 512 targets, we leak the next 9 bits of entropy. Thus, we are able to recover 42 bits of the secret value.

Leaking the last 6 bits: To leak the final 6 bits, we enable speculative execution to $\mathbf{T}_{\mathbf{secret}}$ (now narrowed down to a single cache line we can fill with executable load instructions). By moving a load of an uncached value to different bytes in that cache line, we can quickly determine exactly which byte in the cache line is executed first, and therefore the exact target of $\mathbf{T}_{\mathbf{secret}}$. By combining these steps, we fully leak any unknown value written into the BPU by our *ScratchBPU* Write.

6.4 Automatically Finding Vulnerable Gadgets

From the previous section, we see that an ideal victim is a process which has an indirect branch $\mathbf{I}_{\mathbf{write}}$ on a register containing a secret value $\mathbf{T}_{\mathbf{secret}}$ under a conditional branch **C**. However, such a sequence of instructions is unlikely to be found in modern cryptographic libraries that are hardened against microarchitectural attacks.

Instead, we focus on the pattern of code similar to our *ScratchBPU* Write primitive. Here, we have an indirect branch $\mathbf{I}_{\mathbf{speculation}}$ at a point in the code where a register $\mathbf{reg}_{\mathbf{secret}}$ holds some secret value. We call such a gadget a **Secret At Dispatch (SAD)** gadget. This new class of gadgets is interesting because they do not rely on any secret dependent loads or branches that previous classes of attacks exploited. As long as a secret value is stored in a register at the site of an indirect jump, it can be leaked. The attacker simply has to hijack this indirect jump and direct execution to another indirect branch that jumps on this secret register. This results in the secret value being written into the BPU to be exfiltrated later. An attacker can easily find jumps on different registers within a binary. This leaves finding a register with a secret value and a nearby indirect branch to hijack as the only attacker requirement. PLT mediated library calls and virtual function dispatches in object oriented code are common sources of such sites.

To help us automatically find such gadgets to exploit, we extended the binary analysis tool InSpectre Gadget [71] to detect SAD gadgets in a binary.

Given the binary, a set of entry addresses, and for each entry a set of registers/symbols designated secret, the tool performs bounded symbolic execution from each entry and reports every indirect branch at which one or more registers hold a secret or secret-derived value. These indirect branches together with that set of registers form the SAD gadgets we are interested in.

Table 2 in the Appendix lists the SAD gadgets we found in OpenSSL's cryptographic library when compiled with `gcc` at different optimization levels.

7 Attacks

In this section, we propose two classes of attacks that use primitives demonstrated in the previous section. The first is a covert channel across processes that do not share an address space. The second is to demonstrate the existence of a side channel which can leak secret values through a benign function call to a cryptographic routine.

We have demonstrated the vulnerability exists in both BTB and IBP. The tradeoffs are the following. The IBP encodes 48 bits per entry, while the BTB stores only the lower 32 bits. However, creating aliasing is far easier in the BTB because it is indexed with the PC, while the IBP is indexed with the far more dynamic PHR. Further, in the check and read primitives the attacker has control of the high 16 bits (taken from the current PC) so it has more control over the region of memory where a predicted branch target might land.

Because we have the tools to track and set the PHR, and because the diversity of the PHR gives us a clearer success rate (Section 3.1), we focus on the IBP for these attacks to demonstrate the more difficult, but more powerful version.

7.1 Covert Channel

This section demonstrates how the misuse of the BPU as a scratchpad memory can be used to create a covert channel. A covert channel allows two entities that are otherwise isolated by the system to communicate with each other. We have shown in previous sections how a value written to the BPU in one process can be read across the process boundary. We thus use the *ScratchBPU* Write and *ScratchBPU* Check primitives to quickly encode bits of information across two processes, even with IBPB on. In situations where the IBPB defense is switched off, two untrusting VMs can also use the same mechanism to communicate.

To establish the covert channel, the read process agrees to reserve one page (usually 4KB) in its code section. The write process will use values inside this address range to encode information and write into the BPU. Both processes agree to have a common PC address and a known PHR value which will be used by $\mathbf{I}_{\mathbf{write}}$ and $\mathbf{I}_{\mathbf{read}}$ respectively so that both these branches index into the same BPU entry. To encode N (up to six bits), the sending process uses *ScratchBPU* Write to the address $page_start + N * 64$, so that each unique value of N goes to a different cache line.

The covert channel read process then executes the multiple-value version of *ScratchBPU* Check, where we flush the page from cache, execute an aliased indirect branch once, then simply check the 64 possible cache lines for a hit in the instruction cache. Note that although the last step of the attack is a series of flush and reloads [76], it all happens within the attacker; this attack succeeds even in the presence of cache partitioning mitigations [23, 40].

To measure the bandwidth of our covert channel, we randomly generate 1024 bytes and transmit it. By transmitting each value three times we find that the receiver can decode the value without an error. The transmission rate we achieve this way is **18,483 bps**. By choosing to transmit the value only

once, we can achieve a transmission bandwidth of **55,375 bps**, while incurring 13 bit errors, giving us a bit error rate of $\mathbf{1.58 \times 10^{-3}}$.

7.2 Side Channel

A more powerful use of the *ScratchBPU* primitives is to use it to construct a side channel that attacks unsuspecting victim processes to leak sensitive information to an attacker process.

In our attack, we target the function `PKCS5_PBKDF2_HMAC` which derives a key from a password by using the password as the HMAC key and iterating HMAC over a salted counter thousands of times. Earlier work has demonstrated that branch behavior and cryptographic implementations can expose secret-dependent information [7–9, 59].

In the OpenSSL implementation of this function, we notice that the function calls the libc function `strlen`. Since libc is dynamically linked to binaries by default, the call to `strlen` is resolved through a PLT stub, which performs an indirect jump using the function's address stored in the Global Offset Table (GOT). This GOT entry serves as indirect jump $\mathbf{I_{speculation}}$ for our write primitive.

At the point of this jump, the pointer to the user's plain text password is stored in register `rdi`. We use BTI to cause $\mathbf{I_{speculation}}$ to mispredict towards a piece of code within the OpenSSL library that first de-references `rdi` and then jumps on the de-referenced value. This second jump serves as our $\mathbf{I_{write}}$ that ultimately writes the secret value into the BPU.

We then use the *ScratchBPU* Read primitive to exfiltrate this written value.

As an example, we demonstrate the exfiltration of a 32 bit value `0x12345678` that was written to the IBP. Since we assume this is known to be a 32 bit value, we can bypass Page Table Level 4 (bits 47 to 39 are all 0). The first 7 bits corresponding to Page Table Level 3 (bits 38 to 32) will also be 0, leaving bits 31 and 30 and giving us 4 possible outcomes for that table.

Here is where we invoke Peek-a-Walk [69]. Starting with 00 (then moving to 01, 10, 11 until one of them produces a signal), we map a region whose bits 47 to 30 are 0. Then, after priming and probing the Level 2 Page Table cache, we get a hit on cache line 18. Getting a hit here means that the last two bits of Level 3 are 00 (so we do not have to try the other combinations), and that the next 6 bits corresponding to the Level 2 table are 0x12 (we do not get the last three bits of Level 2 due to cache line granularity). To keep the attacker's own execution out of our measurement, we mask the Page Table Level 2 lines whose entries map to the attacker's own pages. Figure 8 shows an example heatmap showing the signal from line 18. The lines in grey are the masked ones.

Continuing this process, we now map the regions whose high bits are 0x12000000. Masking the attacker's lines is not feasible at this level as the binary may span many lines, hence we rely on a different mechanism. We set the last 3 bits

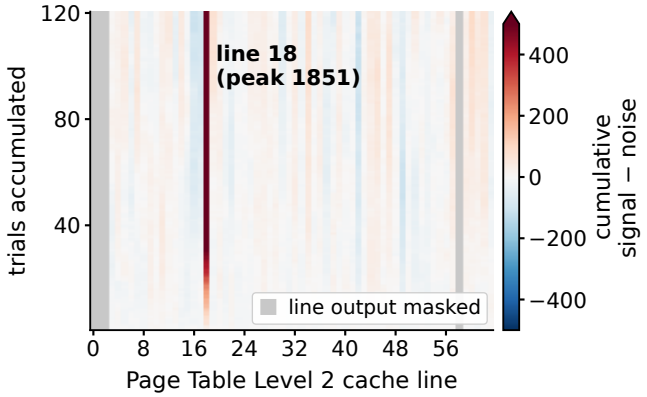


Figure 8: Heatmap of the cache hits when discovering the Page Table Level 2 Entry. We have masked the cache lines belonging to the attacker.

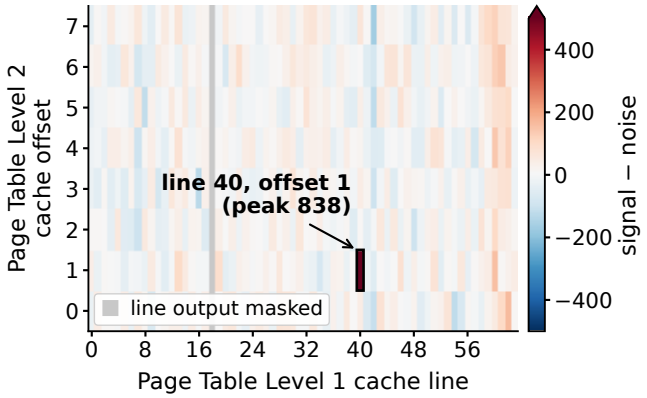


Figure 9: Heatmap of the cache when discovering the Page Table Level 1 Entry. We have masked the cache line for Page Table Level 2 after it is discovered, to prevent interference.

corresponding to Page Table Level 2 to each value between 0 and 7 in turn and repeat the prime and probe. Across the resulting signals, a line falls into one of three categories. A line that never produces a signal was never fetched. A line that produces a signal at every Page Table Level 2 cache offset value belongs to the attacker's own binary, and is noise from the attacker's execution rather than from the victim. The signal produced by the speculative jump on our secret value is the line that produces a signal at exactly one Page Table Level 2 cache offset value. In our example, line 40 is the only line that produces a signal only when the Page Table Level 2 cache offset value is 1. This gives us the upper 17 bits of the secret value. Figure 9 shows this sweep.

Now that we have significantly narrowed down the search space, we flush and reload the I Cache for the next 9 bits (see Figure 13a in the Appendix) to arrive at 0x12345640. Finally, we use our sliding byte technique (see Figure 13b in the Appendix) to leak the last 6 bits (the offset into a cache line) for the full 32 bit address of `0x12345678`.

This end to end attack demonstrates how *ScratchBPU* primitives serve as a powerful tool to leak secret data even from cryptographic libraries that are otherwise hardened against microarchitectural side channel attacks [13].

8 Mitigation

To eliminate the speculative storage vectors established by this work, we can investigate both software (i.e., current) mitigations or hardware (future) mitigations.

Possible software mitigations include enabling strict IBPB in all situations to force the BPU to be flushed in all context switches. Another alternative is for secure workloads to issue BPU flush operations before every indirect jump operation. Using retpolines [3, 4, 62] is another way to break *ScratchBPU* based attacks since retpolines prevent speculative indirect jumps from executing in the first place. However, other exploits like Retbleed [72] already compromise retpoline based defenses.

However, this vulnerability is hardware-induced and we would strongly propose a hardware solution, and therefore choose to investigate such a solution in detail [68, 79, 81, 82]. We propose delaying all BPU updates to the point of architectural commit. Buffering these updates and applying them only when a branch reaches the head of the Reorder Buffer (ROB) ensures that the predictor state is trained exclusively by the architecturally committed instruction stream, rendering the BPU effectively invisible to wrong-path execution.

The conventional reliance on speculative updates is intended to minimize recurrence latency in tight loops and provide a "warm-up" mechanism for branches encountered on the wrong path. In theory, this early exposure reduces cold-start penalties for branches that eventually appear in the committed instruction stream. However, in modern wide-issue pipelines, these perceived benefits are often eclipsed by significant architectural noise [12]. As we demonstrate empirically in Section 8.2, speculative updates to the BTB frequently lead to *ghost branch* pollution – where invalid instructions evict high-value architectural entries – while speculative CBP/IBP updates introduce aliasing noise that forces the predictor to unlearn irrelevant patterns. By deferring updates to commit, we can eliminate this interference and recover lost capacity.

Furthermore, this mitigation is entirely hardware-based and software-transparent, requiring no modifications to the ISA, compiler, or operating system. The implementation also requires minimal modifications to the core's micro-architecture as it leverages the branch checkpoints and metadata already maintained in the ROB for pipeline recovery [10,44,67,70,75]. While serializing updates increases the residency of this metadata, this is easily managed by modestly scaling the depth of existing update queues. Ultimately, our approach represents a pragmatic, low-cost correction that secures the BPU while maintaining, and often improving, architectural performance.

8.1 Experimental Methodology

We evaluate the proposed mitigation using the gem5 architectural simulator [14] in Full-System (FS) mode with an x86-64 Ubuntu 22.04 environment. We model an 8-wide, high performance core emulating an Intel Alder Lake-class architecture operating in single-threaded mode. Full details of the simulated architecture appear in the Appendix in Table 3. The Branch Prediction Unit (BPU) configuration is derived from recent reverse-engineering studies of modern high-performance core [33, 78], featuring a TAGE-based conditional predictor and an ITTAGE-based indirect predictor.

Workload Selection: We benchmark the full SPEC2017 suite [57], including both SPECInt and SPECFP workloads. This selection allows us to evaluate the mitigation across a spectrum of front-end-heavy (branch-intensive) and back-end-heavy (compute-bound) tasks, ensuring a comprehensive assessment of performance parity.

Sampling and Simulation Flow: To ensure representative steady-state measurements, we boot the system using gem5's KVM-based acceleration, fast-forwarding 5 billion cycles to bypass application initialization. Upon switching to the cycle-accurate Out-of-Order (O3) model, we perform a 100-million instruction warmup to prime the cache hierarchy and the core, followed by a 500-million instruction measurement phase.

8.2 Performance Analysis

We evaluate performance across four configurations: *Insecure Baseline* – all BPU components (BTB, CBP, and IBP) are updated speculatively at the resolve stage; *Speculative CBP/IBP Only* – BTB updates are deferred to commit, while CBP and IBP update at resolve; *Speculative BTB Only* – CBP and IBP updates are deferred to commit, while the BTB updates at resolve; *Proposed Mitigation (No Speculative Updates)* – all BPU components are updated exclusively at the point of architectural commit.

Figure 10 (a) illustrates the IPC impact of these configurations relative to the baseline. While the average performance shift across all configurations is marginal, a consistent and notable trend emerges: all three non-baseline configurations outperform the insecure baseline. This result not only reinforces our hypothesis that speculative updates have become vestigial in modern wide-issue pipelines but also reveals some critical insights into branch predictor dynamics.

Speculative CBP/IBP Only: As the primary structure in the BPU hierarchy, the BTB acts as a gatekeeper; a BTB miss prevents subsequent predictors from observing or training on a branch. By serializing BTB updates at commit, the structure is restricted to architecturally valid branches. This creates a high-fidelity filter: the CBP and IBP can still leverage the benefits of speculative updates, but only for branches confirmed to exist on the correct path. This architecture retains the gains of speculative training while eliminating the *ghost branch*

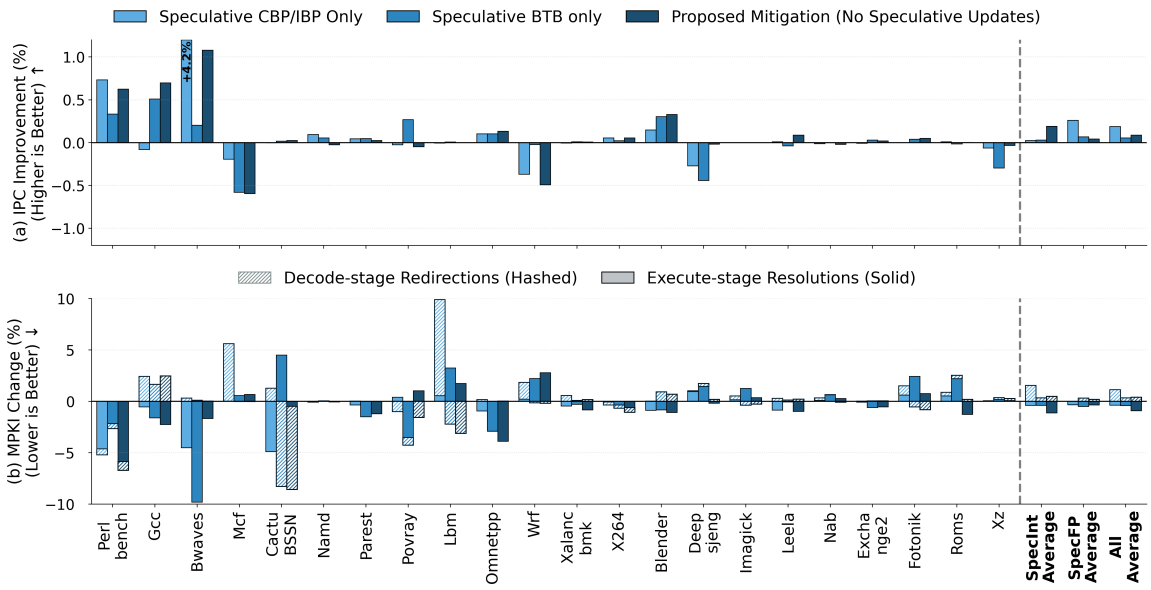


Figure 10: Performance and prediction accuracy impact of various BPU update policies relative to an insecure baseline. Sub-figure (a) shows percentage change in IPC and (b) shows percentage change in total MPKI across the evaluated configurations.

noise introduced by wrong-path execution. The highest performance gain (+0.16% IPC) occurs when only the BTB is restricted to commit-time updates.

Proposed Mitigation: This approach effectively eliminates all speculative poisoning vectors while maintaining performance parity with, and often exceeding, the insecure baseline. While this approach forfeits all potential benefits of speculative pre-training, it achieves the highest average performance gains for *SPECInt* workloads. These integer applications are characterized by high control-flow density, which places significant pressure on BPU capacity. Under such conditions, the BPU is highly sensitive to wrong-path noise. Serializing all BPU updates at commit is the most effective policy for both security and performance in branch-heavy workloads.

8.3 Misprediction Taxonomy and Analysis

To analyze the underlying drivers of IPC change, we categorize mispredictions based on their point of detection. *Decode-stage redirections* occur when the front-end logic detects a missing branch entry, a non-branch instruction incorrectly identified as a branch, or a target mismatch for direct branches. Because these are identified early, the core can resteer the fetch unit without a full backend pipeline flush, making them low-penalty events. In contrast, *Execute-stage resolutions* occur when a conditional branch direction or an indirect branch target is resolved. These represent high-penalty events that require a total pipeline flush.

As shown in Figure 10 (b), all secure configurations result in an average increase in decode-stage redirections. This is a direct consequence of deferred updates: preventing speculative BTB updates reduces the predictor's ability to identify branches encountered during mispeculation, while deferring CBP updates can lead to direction mismatches based on stale or aliased state. However, this increase in early-stage noise is consistently eclipsed by a reduction in high-penalty *execute-stage resolutions*. Across all tested approaches, the reduction in backend flushes demonstrates that the benefit of removing destructive noise (pollution) from the predictor state far outweighs the perceived advantage of speculative training. This shows that for modern deep-window architectures, the *cost* of speculative serialization is primarily paid in low-penalty resteers, while the *gain* is realized in the prevention of expensive pipeline flushes.

9 Conclusion

This work, *ScratchBPU*, demonstrates a new vulnerability enabled by speculative writes to the BTB and the IBP of the Intel branch prediction unit; specifically, that a branch that resolves while speculative and then is squashed will write its target to the BPU. We demonstrate the conditions under which the write happens, and under what conditions it persists long enough to be read as a prediction by another indirect jump. This paper demonstrates that this enables the BPU to be used as cross process data storage for covert and side channels. We demonstrate full attacks, including a side channel that leaks a sensitive register in a cryptographic function, using a gadget enabling the attacker to read a random value from the BPU. Finally, this work demonstrates that this vulnerability can be fully mitigated by a small modification to future branch prediction hardware with no adverse performance effect.

References

[1] Intel® performance monitoring units (pmus). https://perfmon-events.intel.com/#.

[2] Intel single thread indirect branch predictors (stibp). https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/single-thread-indirect-branch-predictors.html, 2018.

[3] Retpoline: a software construct for preventing branch-target-injection. https://support.google.com/faqs/answer/7625886, 2018.

[4] Retpoline: A branch target injection mitigation. https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/retpoline-branch-target-injection-mitigation.html, 2022.

[5] Intel indirect branch predictor barrier (ibpb). https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/indirect-branch-predictor-barrier.html, 2023.

[6] Intel indirect branch restricted speculation (ibrs). https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/indirect-branch-restricted-speculation.html, 2023.

[7] Onur Acıiçmez, Shay Gueron, and Jean-Pierre Seifert. New branch prediction vulnerabilities in openssl and necessary software countermeasures. In *Cryptography and Coding*, 2007.

[8] Onur Aciiçmez, Çetin Kaya Koç, and Jean-Pierre Seifert. On the power of simple branch prediction analysis. In *Computer and Communications Security (CCS)*, 2007.

[9] Onur Acıiçmez, Çetin Kaya Koç, and Jean-Pierre Seifert. Predicting secret keys via branch prediction. In *RSA Conference*, 2007.

[10] Sam Ainsworth and Timothy M Jones. Muontrap: Preventing cross-domain spectre-like attacks by capturing speculative state. In *International Symposium on Computer Architecture (ISCA)*, 2020.

[11] Enrico Barberis, Pietro Frigo, Marius Muench, Herbert Bos, and Cristiano Giuffrida. Branch history injection: On the effectiveness of hardware mitigations against {Cross-Privilege} spectre-v2 attacks. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 971–988, 2022.

[12] Jonathan Behrens, Adam Belay, and M. Frans Kaashoek. Performance evolution of mitigating transient execution attacks. In *European Conference on Computer Systems*, 2022.

[13] Atri Bhattacharyya, Alexandra Sandulescu, Matthias Neugschwandtner, Alessandro Sorniotti, Babak Falsafi, Mathias Payer, and Anil Kurmus. Smotherspectre: exploiting speculative execution through port contention. In *Computer and Communications Security (CCS)*, 2019.

[14] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *SIGARCH Comput. Archit. News*, 39(2):1–7, August 2011. doi:10.1145/2024716.2024718.

[15] Tyler Bletsch, Xuxian Jiang, Vince W. Freeh, and Zhenkai Liang. Jump-oriented programming: A new class of code-reuse attack. In *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011.

[16] Claudio Canella, Jo Van Bulck, Michael Schwarz, Moritz Lipp, Benjamin Von Berg, Philipp Ortner, Frank Piessens, Dmitry Evtyushkin, and Daniel Gruss. A systematic evaluation of transient execution attacks and defenses. In *USENIX Security Symposium (USENIX Security)*, 2019.

[17] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H Lai. Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution. In *IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019.

[18] Dmitry Evtyushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. Jump over aslr: Attacking branch predictors to bypass aslr. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–13. IEEE, 2016.

[19] Dmitry Evtyushkin, Ryan Riley, Nael CSE Abu-Ghazaleh, ECE, and Dmitry Ponomarev. Branchscope: A new side-channel attack on directional branch predictor. In *Proceedings of the Twenty-Third International*

Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '18, 2018.

[20] Agner Fog. Test programs for measuring clock cycles and performance monitoring. https://www.agner.org/optimize/testp.zip, 2025. Accessed: 2026-08-25.

[21] Erik Göktaş, Elias Athanasopoulos, Herbert Bos, and Cristiano Giuffrida. Out of control: Overcoming control-flow integrity. In *Proceedings of the 35th IEEE Symposium on Security and Privacy (IEEE S&P)*, 2014.

[22] Ben Gras, Kaveh Razavi, Erik Bosman, Herbert Bos, and Cristiano Giuffrida. Aslr on the line: Practical cache attacks on the mmu. In *NDSS*, volume 17, page 26, 2017.

[23] Intel Corporation. Intel® Xeon® processor scalable family technical overview. https://www.intel.com/content/www/us/en/developer/articles/technical/xeon-processor-scalable-family-technical-overview.html. Intel Developer Articles. Accessed: April 29, 2026.

[24] Intel Corporation. Speculative execution side channel mitigations. https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/speculative-execution-side-channel-mitigations.html, May 2021. Revision 2.0. Document ID 758363. Accessed: April 24, 2026.

[25] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*, April 2024.

[26] Intel Corporation. PerfMon Events Documentation. https://perfmon-events.intel.com, 2025. Accessed: April 24, 2026.

[27] Md Hafizul Islam Chowdhuryy, Hang Liu, and Fan Yao. Branchspec: Information leakage attacks exploiting speculative branch instruction executions. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*, pages 529–536, 2020. doi:10.1109/ICCD50377.2020.00095.

[28] Daniel A Jiménez and Calvin Lin. Dynamic branch prediction with perceptrons. In *International Symposium on High Performance Computer Architecture (HPCA)*, pages 197–206, 2001.

[29] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. Spectre attacks: Exploiting speculative execution. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019.

[30] Esmaeil Mohammadian Koruyeh, Khaled N Khasawneh, Chengyu Song, and Nael Abu-Ghazaleh. Spectre returns! speculation attacks using the return stack buffer. In *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.

[31] Johnny KF Lee and Alan Jay Smith. Branch prediction strategies and branch target buffer design. *Computer*, 17(01):6–22, 1984.

[32] Sangho Lee, Ming-Wei Shih, Prasun Gera, Taesoo Kim, Hyesoon Kim, and Marcus Peinado. Inferring fine-grained control flow inside {SGX} enclaves with branch shadowing. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 557–574, 2017.

[33] Luyi Li, Hosein Yavarzadeh, and Dean M. Tullsen. Indirector: High-precision branch target injection attacks exploiting the indirect branch predictor. In Davide Balzarotti and Wenyuan Xu, editors, *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. USENIX Association, 2024. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/li-luyi.

[34] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown. *arXiv preprint arXiv:1801.01207*, 2018.

[35] Giorgi Maisuradze and Christian Rossow. ret2spec: Speculative execution using return stack buffers. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 2109–2122, 2018.

[36] Giorgi Maisuradze and Christian Rossow. ret2spec: Speculative execution using return stack buffers. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 2109–2122, 2018. doi:10.1145/3243734.3243761.

[37] Scott McFarling. Combining branch predictors. Technical report, Citeseer, 1993.

[38] Pierre Michaud. A ppm-like, tag-based branch predictor. *JILP-Championship Branch Prediction*, 7:10, 2005.

[39] Milena Milenkovic, Aleksandar Milenkovic, and Jeffrey Kulick. Demystifying intel branch predictors. In *Workshop on Duplicating, Deconstructing and Debunking*, 2002.

[40] Khang T. Nguyen. Introduction to cache allocation technology in the Intel® Xeon® processor E5 v4 family. https://www.intel.com/content/www/us/en/developer/articles/technical/

introduction-to-cache-allocation-technology.html, February 2016. Intel Developer Articles. Article ID: 672631. Accessed: April 29, 2026.

[41] Colin Percival. Cache missing for fun and profit, 2005.

[42] Xida Ren, Logan Moody, Mohammadkazem Taram, Matthew Jordan, Dean M. Tullsen, and Ashish Venkat. I see dead µops: Leaking secrets via intel/amd micro-op caches. In *International Symposium on Computer Architecture (ISCA)*, June 2021.

[43] Sandro Rüegge, Kaveh Razavi, and Johannes Wikner. Branch privilege injection: compromising spectre v2 hardware mitigations by exploiting branch predictor race conditions. In *Proceedings of the 34th USENIX Conference on Security Symposium*, USA, 2025. USENIX Association.

[44] Gururaj Saileshwar and Moinuddin K Qureshi. Cleanupspec: An "undo" approach to safe speculation. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

[45] Michael Schwarz, Martin Schwarzl, Moritz Lipp, Jon Masters, and Daniel Gruss. Netspectre: Read arbitrary memory over network. In *European Symposium on Research in Computer Security*. Springer, 2019.

[46] Andre Seznec. The o-gehl branch predictor. *JILP-Championship Branch Prediction*, 2004.

[47] André Seznec. A 64-kbytes ittage indirect branch predictor. In *JWAC-2: Championship Branch Prediction*, 2011.

[48] André Seznec. A new case for the tage branch predictor. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 117–127, 2011.

[49] André Seznec. Tage-sc-l branch predictors. In *JILP-Championship Branch Prediction*, 2014.

[50] André Seznec and Pierre Michaud. A case for (partially) tagged geometric history length branch prediction. *The Journal of Instruction-Level Parallelism*, 8:23, 2006.

[51] André Seznec and Pierre Michaud. A case for (partially) tagged geometric history length branch prediction. *JILP-Championship Branch Prediction*, 2006.

[52] Hovav Shacham. The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86). In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, 2007.

[53] Vedvyas Shanbhogue, Deepak Gupta, and Ravi Sahita. Security analysis of processor instruction set architecture for enforcing control-flow integrity. In *Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy*, HASP '19, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3337167.3337175.

[54] Basavesh Ammanaghatta Shivakumar, Jack Barnes, Gilles Barthe, Sunjay Cauligi, Chitchanok Chuengsatiansup, Daniel Genkin, Sioli O'Connell, Peter Schwabe, Rui Qi Sim, and Yuval Yarom. Spectre declassified: Reading from the right place at the wrong time. In *IEEE Symposium on Security and Privacy (SP)*, 2023.

[55] James E. Smith. A study of branch prediction strategies. In *International Symposium on Computer Architecture (ISCA)*, page 135–148, 1981.

[56] Spectre side channels. https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/spectre.html, 2019.

[57] Standard Performance Evaluation Corporation. *SPEC CPU 2017 Benchmark Suite*, 2017. https://www.spec.org/cpu2017/.

[58] Mohammadkazem Taram, Xida Ren, Ashish Venkat, and Dean Tullsen. Secsmt: Securing SMT processors against contention-based covert channels. In *USENIX Security Symposium (USENIX Security)*, 2022.

[59] Eran Tromer, Dag Arne Osvik, and Adi Shamir. Efficient cache attacks on AES, and countermeasures. *Journal of Cryptology*, 2010.

[60] Daniël Trujillo, Johannes Wikner, and Kaveh Razavi. Inception: Exposing new attack surfaces with training in transient execution. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 7303–7320, 2023.

[61] Dean M Tullsen, Susan J Eggers, and Henry M Levy. Simultaneous multithreading: Maximizing on-chip parallelism. In *International Symposium on Computer Architecture (ISCA)*, 1995.

[62] Paul Turner. Retpoline: A software construct for preventing branch-target-injection. https://support.google.com/faqs/answer/7625886, 2018. Google Help. Accessed: April 29, 2026.

[63] Vladimir Uzelac. Microbenchmarks and mechanisms for reverse engineering of modern branch predictor units. *A Masters Thesis submitted to the University of Alabama*, 2008.

[64] Vladimir Uzelac and Aleksandar Milenkovic. Experiment flows and microbenchmarks for reverse engineering of branch predictor structures. In *International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009.

[65] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the intel SGX kingdom with transient Out-of-Order execution. In *USENIX Security Symposium (USENIX Security)*, 2018.

[66] Stephan Van Schaik, Kaveh Razavi, Ben Gras, Herbert Bos, and Cristiano Giuffrida. RevAnC: A framework for reverse engineering hardware page table caches. In *Proceedings of the European Workshop on Systems Security (EuroSec)*, 2017.

[67] Marco Vassena, Craig Disselkoen, Klaus V Gleissenthall, Sunjay Cauligi, Rami Gökhan Kici, Ranjit Jhala, Dean Tullsen, and Deian Stefan. Automatically eliminating speculative leaks from cryptographic code with Blade. In *Principles of Programming Languages (POPL)*, 2021.

[68] Ilias Vougioukas, Nikos Nikoleris, Andreas Sandberg, Stephan Diestelhorst, Bashir M Al-Hashimi, and Geoff V Merrett. Brb: Mitigating branch predictor side-channels. In *International Symposium on High Performance Computer Architecture (HPCA)*, 2019.

[69] Alan Wang, Boru Chen, Yingchen Wang, Christopher W. Fletcher, Daniel Genkin, David Kohlbrenner, and Riccardo Paccagnella. Peek-a-walk: Leaking secrets via page walk side channels. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3534–3548, 2025. doi:10.1109/SP61157.2025.00023.

[70] Ofir Weisse, Ian Neal, Kevin Loughlin, Thomas F Wenisch, and Baris Kasikci. Nda: Preventing speculative execution attacks at their source. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

[71] Sander Wiebing, Alvise de Faveri Tron, Herbert Bos, and Cristiano Giuffrida. {InSpectre} gadget: Inspecting the residual attack surface of cross-privilege spectre v2. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 577–594, 2024.

[72] Johannes Wikner and Kaveh Razavi. {RETBLEED}: Arbitrary speculative code execution with return instructions. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3825–3842, 2022.

[73] Johannes Wikner, Daniël Trujillo, and Kaveh Razavi. Phantom: Exploiting decoder-detectable mispredictions. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.

[74] Henry Wong. Microbenchmarking return address branch prediction, apr 2018. URL: http://blog.stuffedcow.net/2018/04/ras-microbenchmarks.

[75] Mengjia Yan, Jiho Choi, Dimitrios Skarlatos, Adam Morrison, Christopher Fletcher, and Josep Torrellas. Invisispec: Making speculative execution invisible in the cache hierarchy. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018.

[76] Yuval Yarom and Katrina Falkner. Flush+reload: A high resolution, low noise, l3 cache side-channel attack. In *23rd USENIX Security Symposium (USENIX Security 14)*, pages 719–732. USENIX Association, 2014.

[77] Hosein Yavarzadeh, Archit Agarwal, Max Christman, Christina Garman, Daniel Genkin, Andrew Kwong, Daniel Moghimi, Deian Stefan, Kazem Taram, and Dean Tullsen. Pathfinder: High-resolution control-flow attacks exploiting the conditional branch predictor. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[78] Hosein Yavarzadeh, Mohammadkazem Taram, Shravan Narayan, Deian Stefan, and Dean M. Tullsen. Half&half: Demystifying intel's directional branch predictors for fast, secure partitioned execution. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 1220–1237. IEEE, 2023. doi:10.1109/SP46215.2023.10179415.

[79] Tao Zhang, Timothy Lesch, Kenneth Koltermann, and Dmitry Evtyushkin. Stbpu: A reasonably secure branch prediction unit. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2022.

[80] Zhiyuan Zhang, Mingtian Tao, Sioli O'Connell, Chitchanok Chuengsatiansup, Daniel Genkin, and Yuval Yarom. BunnyHop: Exploiting the instruction prefetcher. In *USENIX Security Symposium (USENIX Security)*, 2023.

[81] Lutan Zhao, Peinan Li, Rui Hou, Michael C Huang, Jiazhen Li, Lixin Zhang, Xuehai Qian, and Dan Meng. A lightweight isolation mechanism for secure branch predictors. In *Design Automation Conference (DAC)*, 2021.

[82] Lutan Zhao, Peinan Li, Rui Hou, Michael C Huang, Xuehai Qian, Lixin Zhang, and Dan Meng. Hybp: Hybrid

isolation-randomization secure branch predictor. In *International Symposium on High Performance Computer Architecture (HPCA)*, 2022.

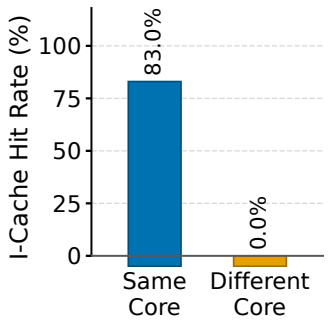


Figure 11: Variation of I Cache Hit Percentage Across Process Boundary when Write and Read Process Run on Same vs Different Cores.

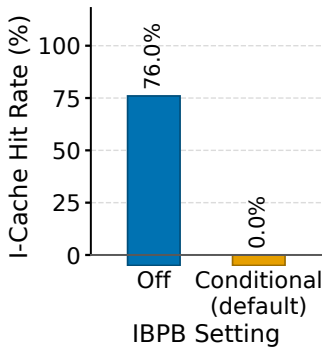


Figure 12: Variation of I Cache Hit Percentage Across VM Boundary when IBPB is turned off vs when IBPB is in conditional mode (default).

A Ethical Considerations

This work aims to advance understanding of the security implications of high performance microarchitectural optimizations by identifying fundamental weaknesses in isolation. All experiments are conducted in controlled laboratory environments on machines owned or explicitly authorized by the authors and collaborating vendors. We responsibly disclosed our findings to Intel prior to submission.

We are confident that this research complies with all applicable legal and ethical standards. The study is conducted in the interest of improving global computer system security and adheres to the Usenix 2027 Ethics Guidelines.

This paper was edited only for spelling and grammar using OpenAI ChatGPT and Google Gemini.

B Open Science

We release an anonymized artifact repository containing scripts, configuration files, and data analysis code used to generate the experimental results reported in this paper. The code and artifacts can be found at https://anonymous.4open.science/r/Usenix-Submission-1007/README.md The zip file is uploaded to HotCRP.

C Testing Intel STIBP Defense

Figure 11 shows the variation of I Cache Hit rate when the write and read processes run on the same core and different cores. This result tells us that the attacker and victim must be co-located on the same logical core.

D Testing Intel IBPB Defense

Figure 12 shows the variation of I Cache Hit rate when the write process and read process run on different VMs. This result tells us that when the IBPB setting is set to the default conditional value, the BPU is flushed either on `VMENTER` or `VMEXIT`.

E Gadget Listing

Listing 5 shows the set of instructions in the OpenSSL `PKCS5_PBKDF2_HMAC` function that we exploit in our side channel attack. A pointer to the user's plaintext password is stored in the register `rdi` and a call to `strlen` is made which is handled with a PLT stub and GOT lookup.

```
; --- caller site (5f1b03h) ---
    mov rdi, 0xf3a640      ; pass   -> user
    passphrase
    mov rsi, -1            ; passlen = -1 forces
    the strlen path
    mov rdx, 0xf3a648      ; salt
    mov ecx, 0xb           ; saltlen
    mov r8d, 0x2710        ; iterations
    push 0xf3a654          ; out
    push 0x20              ; keylen
    call PKCS5_PBKDF2_HMAC

; --- PKCS5_PBKDF2_HMAC (407b10h) ---
PKCS5_PBKDF2_HMAC:
    sub  rsp, 0x28
    mov  r10d, [rsp + 0x30]       ; keylen
    mov  rax, [rsp + 0x38]        ; out
    call ossl_pkcs5_pbkdf2_hmac_ex

; --- ossl_pkcs5_pbkdf2_hmac_ex (4078d0h) ---
ossl_pkcs5_pbkdf2_hmac_ex:
    mov  r13, rdi          ; r13 = pass, kept
    across the call
    mov  r12d, esi         ; r12d = passlen
    ...
    test r13, r13
    je   .skip
    cmp  r12d, -1
    jne  .have_len
    mov  rdi, r13          ; rdi = pass
    call strlen@plt        ; I_speculation: rdi and
     r13 both hold pass

; --- strlen@plt (403998h) ---
strlen@plt:
    jmp qword [rip + 0x33061a]   ; -> GOT[strlen]
    (733fb8h)
```

Listing 5: OpenSSL `PKCS5_PBKDF2_HMAC` gadget. Passing -1 as `passlen` forces the `strlen` path, and the PLT stub for `strlen` is an indirect jump through the GOT with the passphrase pointer live in `rdi`.

F OpenSSL ScratchBPU Gadgets

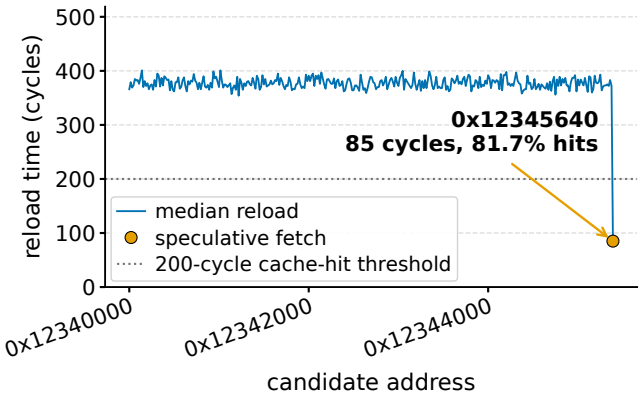
Table 2 lists all the Secret At Dispatch gadgets we found with our automatic gadget finding tool. We bounded the number of basic blocks explored to 10 and compiled the binary with `gcc` at different optimization levels.

G Simulation Details of Mitigation Experiments

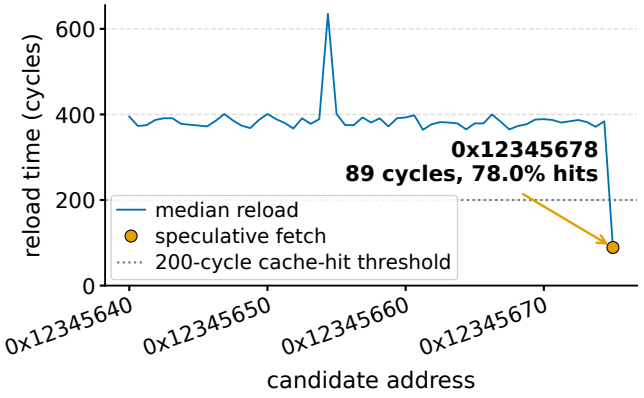
Table 3 lists the configuration of the processor we used to simulate our mitigation results in gem5.

H Recovering the Lower Bits of the Secret Value

Figure 13a shows the variation of I Cache reload time of our test target when it is placed at different addresses in 64 byte increments. Figure 13b shows the variation of I Cache reload time as the load instruction is slid byte by byte across the discovered cache line. The reload time drops at the first offset where speculation lands at or before the load, revealing the final 6 bits and giving the full address.



(a) Sweeping candidate addresses to resolve bits 14–6, which narrows the secret to the cache line at `0x12345640`.



(b) Sliding the load instruction across that cache line to resolve the final 6 bits, giving the full address `0x12345678`.

Figure 13: Recovering the low bits of the secret address by flush and reload. In both stages the median I Cache reload time stays near 400 cycles while the candidate address is wrong; when it matches the value written into the BPU, the speculative fetch pulls the line back and the reload time drops below the 200-cycle cache-hit threshold.

Table 2: Secret At Dispatch (SAD) gadgets found in OpenSSL `libcrypto.so.3` across GCC optimization levels O0–O3. The symbolic execution was capped to exploring a maximum of 10 basic blocks

Function	**Secret (Registers)**	**Optimization Levels**
`SRP_Calc_x/_ex`	Password (`rdx`, `r13`)	O1, O2, O3
`SRP_create_verifier_ex`	Password (`rdx`, `rax`, `r13`)	O1, O2, O3
`SRP_create_verifier_BN/_BN_ex`	Password (`r13`)	O1
`SRP_create_verifier`	Password (`r13`)	O3
`CMS_decrypt_set1_password`	Password (`rdi`, `rsi`, `rdx`, `rbx`, `rbp`)	O0, O1, O2, O3
`EVP_MAC_init`	MAC key (`rdi`, `rsi`, `r8`, `rax`)	O0, O1, O2, O3
`HMAC`	HMAC key (`rsi`, `rbx`)	O0, O1, O2, O3
`HMAC_Init/_ex`	HMAC key (`rsi`, `rbx`, `r13`, `r14`, `r15`)	O0, O1, O2, O3
`EVP_EncryptInit_ex/_DecryptInit_ex`	Cipher key (`rdi`, `rcx`, `rax`, `rbx`)	O0, O1, O2, O3
`EVP_CipherInit_ex`	Cipher key (`rdi`, `rcx`, `rax`, `rbx`, `r13`, `r14`)	O0, O1, O2, O3
`DES_string_to_key`	Password (`rdi`, `rbx`, `rbp`)	O1, O2, O3
`DES_string_to_2keys`	Password (`rdi`, `r12`)	O1, O2, O3
`PKCS5_PBKDF2_HMAC`	Password (`rdi`, `rax`, `r12`, `r13`)	O0, O1, O2, O3
`PKCS5_PBE_keyivgen/_ex`	Password (`rsi`, `rax`, `r12`, `r13`)	O0, O1, O2, O3
`EVP_PBE_CipherInit/_ex`	Password (`rsi`, `rdx`, `rax`, `rbx`, `rbp`)	O0, O1, O2, O3
`EVP_PBE_scrypt/_ex`	Password (`rdi`, `rbp`)	O0, O1, O2, O3
`PKCS8_encrypt/_ex`	Passphrase (`rdx`, `rbp`, `r15`)	O0, O1, O2, O3
`PKCS12_key_gen_asc/_asc_ex`	Password (`rdi`, `rax`, `rbx`, `r13`)	O0, O1, O2, O3
`PKCS12_key_gen_utf8/_utf8_ex`	Password (`rdi`, `rax`, `rbx`, `rbp`, `r13`)	O0, O1, O2, O3
`PKCS12_key_gen_uni/_uni_ex`	Password (`rdi`, `r13`)	O0, O2, O3
`PKCS12_gen_mac`	Password (`rdi`, `rsi`, `rdx`, `rax`, `rbx`, `rbp`)	O0, O1, O2, O3
`EVP_PKEY_new_raw_private_key`	Ed25519/X25519 private key (`r9`, `r13`, `r14`)	O0, O1, O2, O3
`EVP_PKEY_new_raw_private_key_ex`	Ed25519/X25519 private key (`rcx`, `r9`, `r12`, `r14`)	O0, O1, O2, O3

Table 3: Baseline System Configuration

Feature	**Configuration**
Issue Width	8-wide (Fetch, Decode, Dispatch)
Reorder Buffer	512 entries
Physical Registers	256 Integer, 256 FP, 256 Vector
Caches and Main Memory	
L1 Instruction Cache	64 KiB, 8-way associative
L1 Data Cache	64 KiB, 8-way associative
L2 Unified Cache	4 MiB, 16-way associative
DRAM	3 GB Dual-channel DDR5-8400
Prefetching - All Levels	Stride-based prefetchers
Branch Prediction Unit	
Branch Target Buffer	12-way with 1024 sets
Conditional Predictor - Base	2^{13}-entry, 2-bit bimodal
Conditional Predictor - TAGE	Three 4-way, 512-set tables
Indirect Predictor - ITTAGE	Three 2-way, 256-set tables